PYTHON PROGRAMMING ASSIGNMENT

ASSIGNMENT 2 – CO2

PYTHON PROGRAM IMPLEMENTATION AND EXECUTION EVIDENCE

Course: CSA0806 – Python Programming

Student Name: Praveen L

Register Number: 1924260137

Department: Artificial Intelligence and Data Science.

Faculty Name: HARIPRIYA

Abstract

This assignment focuses on the application of Python control flow constructs, string operations, functions, and recursion to solve practical problems. It covers decision-making statements such as if, if–else, and if–elif–else, along with looping statements such as while and for. Control statements including break, continue, and pass are used to manage program execution effectively. String manipulation techniques are applied to normalise, validate, and format vehicle registration numbers. The assignment also explores efficient string construction using concatenation and the join() method. Functions with parameters, return values, default arguments, and variable scope are studied through a college canteen order system. Recursion and memoisation are used to calculate the number of ways to climb stairs, and their performance is compared with an iterative approach. Finally, a design challenge evaluates program limitations and introduces improved solutions using execution time, function-call counts, and space complexity. Overall, this assignment develops practical programming, analytical thinking, debugging, and performance evaluation skills using Python.

Contents

1. Introduction
2. System Overview and Requirements
3. Task 1 – Decisions with Many Boundaries
4. Task 2 – Loops that Count Attempts
5. Task 3 – Vehicle Registration Number Validator
6. Task 4 – Building Long Text: Concatenation and Join
7. Task 5 – Functions, Parameters and Scope
8. Task 6 – Recursion and Its Limits
9. Task 7 – Diagnose, Redesign and Validate
10. Performance Analysis and Complexity Comparison
11. Results and Discussion
12. Conclusion
13. References
14. Appendix – Python Source Code and Execution Evidence

2. System Overview and Requirements

2.1 System Overview

This assignment focuses on Python programming concepts such as decision-making statements, loops, string operations, functions, and recursion. Seven tasks are implemented to solve practical problems, including electricity subsidy calculation, OTP verification, vehicle number validation, text construction, canteen order management, and staircase calculations. The programs are tested and compared to improve efficiency and understand their time and space complexity.

2.2 Hardware Requirements

Component	Requirement
Processor	Dual-core processor
RAM	4 GB or above
Storage	1 GB free space
Input/Output	Keyboard and monitor

2.3 Functional Requirements

1. Decision making: Determine electricity subsidy eligibility using conditions and boundary values.
2. Loop control: Verify OTP entries, handle blank input and lock the account after the permitted attempts.
3. String manipulation: Normalise, validate and format vehicle registration numbers.
4. Text construction: Compare string concatenation with the join() method and measure execution time.
5. Function management: Demonstrate parameters, return values, default arguments and variable scope.
6. Recursion: Calculate staircase combinations using plain recursion and memoisation.
7. Performance improvement: Compare an original solution with a redesigned iterative solution.

Task 1 — Decisions with many boundaries

1.1 Explanation :

The program accepts electricity units from the user and calculates the subsidy amount based on the units consumed. It uses if, elif, and else statements to check different unit ranges and assign the appropriate subsidy. Finally, it displays the electricity units and subsidy amount.

Inputs, outputs, constants and data types

Item	Suggested type	Validation / purpose
Electricity units	Integer (int)	Must be a non-negative integer.
Subsidy amount	Integer (int)	Must match the electricity unit range.
User input	String (str)	Must contain a valid integer value before conversion.

1.2 Assumptions:

- The electricity units are entered as a positive integer.
- The subsidy amount depends on the number of electricity units consumed.
- The subsidy values are sample values used for this task.
- If the units are 100 or less, the subsidy is Rs. 0.
- The program uses if, elif, and else statements to select the subsidy amount.
- The user enters a valid numerical input.

1.3Alternative data representations:

Electricity Units	Subsidy Amount
0–100	**Rs. 0**
101–200	**Rs. 50**
201–300	**Rs. 100**
Above 300	**Rs. 150**

The electricity units and subsidy amounts are represented using integers. The unit ranges are 0–100 for Rs. 0, 101–200 for Rs. 50, 201–300 for Rs. 100, and above 300 for Rs. 150. These values can also be represented using a table or flowchart to explain the decision-making process clearly.

1.4 Python Program

```
ask-6(2).py 1 ●  Task-7(2).py  Task-8(2).py  Task-95.py  Task-10(2).py  Task-11(2).py  Task-12(2).py 9+  task 2,1.py ×
C: > Users > HP > Downloads > assign 1 > task 2,1.py > ...
1   units = int(input("Enter electricity units: "))
2
3   if units <= 100:
4       subsidy = 0
5   elif units <= 200:
6       subsidy = 50
7   elif units <= 300:
8       subsidy = 100
9   else:
10      subsidy = 150
11
12  print("Subsidy Amount: Rs.", subsidy)
```

Output:

```
P/Downloads/assign 1/task 2,1.py"
Enter electricity units: 3
Subsidy Amount: Rs. 0
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe
P/Downloads/assign 1/task 2,1.py"
Enter electricity units: 245
Subsidy Amount: Rs. 100
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe
P/Downloads/assign 1/task 2,1.py"
Enter electricity units: 567
Subsidy Amount: Rs. 150
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

Task 2 — Loops that count attempts

The program uses a while loop to allow the user to enter an OTP up to three times. The if condition checks whether the entered OTP matches the correct OTP. If it matches, the program displays a success message. Otherwise, it displays an error message and allows another attempt. After three incorrect attempts, the program displays the maximum attempts message.

2.1 Assumptions

The program assumes that the correct OTP is already stored in the system. The user is allowed a maximum of three attempts to enter the OTP. If the entered OTP matches the stored OTP, verification is successful. Otherwise, the program displays an error message.

2.2 Alternative Data Representation

The OTP is represented as a string because it contains numeric characters and may have leading zeros. The number of attempts is represented as an integer. The verification status is determined using conditional statements.

2.3 Input Validation Table

Item	Suggested Type	Validation
Correct OTP	String (str)	Must contain exactly four digits.
User OTP	String (str)	Must contain exactly four digits.
Attempts	Integer (int)	Must not exceed three attempts.
Verification status	Boolean (bool)	Must be True or False.

Task 3 — VEHICLE REGISTRATION NUMBER VALIDATOR

3.1 Explanation:

This task uses Python string operations and conditional statements to validate a vehicle registration number. The program checks whether the entered registration number follows the expected format.

3.2 Python Program

C: > Users > HP > Downloads > assign 1 > task2,2.py > ...

```python
correct_otp = "1234"
attempts = 0

while attempts < 3:
    otp = input("Enter OTP: ")

    if otp == correct_otp:
        print("OTP verified successfully")
        break
    else:
        attempts += 1
        print("Incorrect OTP")

if attempts == 3:
    print("Maximum attempts reached")
```

Output

```
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/H
P/Downloads/assign 1/task2,2.py"
Enter OTP: 143& C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/Downloads/assign 1/task2,2.py"
Incorrect OTP
Enter OTP: 1234
OTP verified successfully
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

3.3 Sample Output

Enter vehicle registration number: TN10AB1234

Valid vehicle registration number

4.4 Brief Explanation

The program accepts a vehicle registration number and converts it into uppercase using upper(). It checks the length and verifies the letter and digit positions using isalpha() and isdigit(). If all conditions are satisfied, it displays a valid message; otherwise, it displays an invalid message.

3.5 Assumptions

The program assumes that the vehicle registration number contains exactly 10 characters in the format TN10AB1234. The first two characters and the next two letters must follow the specified pattern, while the remaining positions must contain digits.

3.6 Alternative Data Representation

The registration number is stored as a string because it contains both letters and numbers. String slicing is used to separate the different parts of the registration number for validation.

3.7 Input Validation Table

Item	Suggested Type	Validation
Registration number	String (str)	Must contain exactly 10 characters.
State code	String (str)	First two characters must be letters.
RTO code	String (str)	Third and fourth characters must be digits.
Series code	String (str)	Fifth and sixth characters must be letters.
Vehicle number	String (str)	Last four characters must be digits.

3.8 Result

The vehicle registration number validator was successfully implemented using Python string operations and conditional statements. It identifies whether the entered registration number matches the specified format.

Task 4 — BUILDING LONG TEXT: CONCATENATION AND JOIN

4.1 Explanation

This task explains how to combine multiple words or strings into a single sentence using Python. String concatenation and the join() method are used to connect different text values. These operations are useful for preparing messages, reports, and other text-based information.

4.2 Brief Explanation

The program stores five words in a list. The join() method combines all the words into a single sentence. A space is placed between each word. Finally, the print() statement displays the complete sentence.

4.3 Assumptions

1. All elements in the list are strings.
2. The list contains at least one word.
3. The words are arranged in the correct order.

4. A space is used between each word.
5. The program does not require additional input from the user.
6. The final output is displayed as a single sentence.

4.4 Python Program

```
› Users › HP › Downloads › assign 1 › task 2,4.py › ...
1   words = ["Python", "is", "easy", "to", "learn"]
2
3   text = " ".join(words)
4
5   print("Combined Text:", text)
```

Output

```
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/Downloads/assign 1/task 2,4.py"
Combined Text: Python is easy to learn
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

4.5 Alternative Data Representation

The words are stored using a Python list, which allows multiple strings to be maintained together. The final sentence is represented as a string. The join() method combines the list elements, while the + operator can also be used for string concatenation.

4.6 Input Validation Table

Item	Suggested Type	Validation
Words	List (list)	Must contain string elements.
Individual word	String (str)	Must contain valid text.
Separator	String (str)	Must be a valid separator, such as a space.
Combined text	String (str)	Must contain the joined words.

4.7 Applications

1. Creating sentences from individual words.
2. Preparing messages and notifications.

3. Combining user information into a single string.
4. Formatting text for reports.
5. Generating descriptions and summaries.

4.8 Result

The program was successfully implemented using Python string concatenation and the join() method. It combines multiple words into a single sentence and displays the result correctly.

Task 5 — FUNCTIONS, PARAMETERS AND SCOPE

5.1 Explanation

This task demonstrates the use of functions in Python to perform a specific operation. Functions help divide a program into smaller parts and make the code easier to understand, reuse, and maintain. Parameters are used to pass values into a function, and the function returns the calculated result.

5.2 Brief Explanation

The program defines a function named calculate_total() with two parameters: price and quantity. It multiplies these values and returns the total amount. The user enters the item price and quantity, and the function calculates and displays the final bill amount.

5.3 Assumptions

1. The user enters a valid item price.
2. The item price must be greater than zero.
3. The quantity must be a positive integer.
4. The function receives two input parameters.
5. The total amount is calculated by multiplying price and quantity.
6. The function returns the calculated amount.

5.4 Python Program

```
C: > Users > HP > Downloads > assign 1 > task2,5.py > ...
1   def calculate_total(price, quantity):
2       total = price * quantity
3       return total
4
5   price = float(input("Enter item price: "))
6   quantity = int(input("Enter quantity: "))
7
8   amount = calculate_total(price, quantity)
9
10  print("Total Amount: Rs.", amount)
```

Output

```
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/H
P/Downloads/assign 1/task2,5.py"
Enter item price: 2456
Enter quantity: 7
Total Amount: Rs. 17192.0
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

5.5 Alternative Data Representation

The item price is represented using a float data type because it may contain decimal values. The quantity and total calculation are represented using numeric data types. The function uses parameters to receive values and a return statement to provide the result.

5.6 Input Validation Table

Item	Suggested Type	Validation
Item price	Float (float)	Must be greater than zero.
Quantity	Integer (int)	Must be a positive integer.
Total amount	Float (float)	Must be the product of price and quantity.
Function parameters	Numeric	Must receive valid values.

5.7 Important Function Concepts

1. Function Definition: The def keyword is used to create a function.
2. Parameters: Price and quantity are passed into the function.

3. Function Call: The function is called using calculate_total().
4. Return Statement: The return statement sends the calculated result.
5. Local Variables: Variables created inside a function are local to that function.
6. Global Variables: Variables created outside functions can be accessed within functions, subject to Python's scope rules.

5.8 Applications

1. Calculating shopping bills.
2. Preparing student mark calculations.
3. Calculating employee salaries.
4. Performing mathematical operations.
5. Reusing the same code in different parts of a program.

5.9 Result

The program was successfully implemented using Python functions, parameters, and return statements. It calculates the total item price based on the given price and quantity.

TASK 6 – RECURSION AND ITS LIMITS

6.1 Explanation

This task demonstrates recursion in Python. Recursion is a process in which a function calls itself to solve a problem. In this task, a recursive function is used to calculate the factorial of a given number. A base condition is used to stop the function from calling itself continuously. The program uses a recursive function called factorial(). The function calls itself with a smaller number until it reaches 0 or 1. The base condition returns 1 and stops the recursion. Finally, the program displays the factorial result.

6.2 Assumptions

1. The user enters a valid integer.
2. The input number must be zero or positive.
3. Factorial is calculated using multiplication.
4. The base condition is 0 or 1.
5. The recursive function continues until the base condition is reached.
6. Very large numbers may cause recursion depth limitations.

6.3 Alternative Data Representation

The input number and factorial result are represented using integers (int). The recursive function repeatedly calls itself with decreasing values. The base condition controls when the recursion stops.

6.4 Python Program

```
C: > Users > HP > Downloads > assign 1 > task2,6.py > ...
1   def factorial(n):
2       if n == 0 or n == 1:
3           return 1
4       else:
5           return n * factorial(n - 1)
6
7   num = int(input("Enter a number: "))
8
9   if num < 0:
10      print("Factorial is not defined")
11  else:
12      print("Factorial:", factorial(num))
```

output

```
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe
P/Downloads/assign 1/task2,6.py"
Enter a number: 5
Factorial: 120
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

6.5 Input Validation Table

Item	Suggested Type	Validation
Input number	Integer (int)	Must be zero or positive.
Factorial result	Integer (int)	Must be calculated correctly.
Recursive function	Function	Must contain a base condition.
Recursive value	Integer (int)	Must decrease in each recursive call.

6.6 Important Concepts

1. Recursion: A function calls itself.
2. Base Condition: Stops the recursive calls.
3. Recursive Call: Calls the same function with a smaller value.
4. Factorial: Product of all positive integers from 1 to the given number.
5. Recursion Limit: Python has a limit on the number of recursive calls.
6. Stack Memory: Each recursive call uses additional memory.

s6.7 Applications

1. Factorial calculation.
2. Fibonacci series.
3. Tree traversal.
4. Searching algorithms.
5. Solving mathematical problems.

6.8 Result

The factorial program was successfully implemented using recursion in Python. It calculates the factorial of a non-negative integer using a recursive function and a base condition.

TASK 7 – DIAGNOSE, REDESIGN AND VALIDATE

7.1 Explanation

This task focuses on identifying inefficient Python code and improving its performance. The program compares two methods of combining multiple strings: repeated string concatenation using += and the join() method. Both methods are tested to check whether they produce the same output and which method is faster. The program first combines 10,000 words using repeated string concatenation with +=. Then, it combines the same words using the join() method. The time.perf_counter() function measures the execution time of both methods. Finally, the program checks whether both methods produce equal results and compares their execution times.

7.2 Assumptions

1. The input contains 10,000 string elements.
2. All list elements contain valid string values.
3. Both methods must produce the same final text.
4. Execution time is measured using the same computer.
5. The join() method is expected to perform better for large collections of strings.
6. The actual execution time may vary depending on the system.

7.3 Alternative Data Representation

The input data is represented using a Python list containing strings. The combined output is stored as a string. Execution times are represented using floating-point numbers. The equality operator (==) is used to validate the results.

7.4 Python Program:

```
> Users > HP > Downloads > assign 1 > task2,7.py > ...
13  start = time.perf_counter()
14
15  text2 = ";".join(items)
16
17  time2 = time.perf_counter() - start
18
19  if text1 == text2 + ";":
20      print("Both outputs are equal")
21  else:
22      print("Outputs are different")
23
24  print("Original method time:", time1)
25  print("Improved method time:", time2)
26
27  if time2 < time1:
28      print("join() method is faster")
29  else:
30      print("Repeated concatenation is faster")
```

Output

```
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/Downloads/assign 1/task2,7.py"
Both outputs are equal
Original method time: 0.15508749999571592
Improved method time: 0.00019929997506551445
join() method is faster
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code>
```

7. 5 Input Validation Table

Item	Suggested Type	Validation
Items	List (list)	Must contain string values.
Individual item	String (str)	Must contain valid text.
Original output	String (str)	Must contain all input items.
Improved output	String (str)	Must match the original output.
Execution time	Float (float)	Must be a non-negative value.

7.6 Performance Comparison

Method	Description	Performance
+=	Adds strings repeatedly	Can be slower for large strings
join()	Combines strings in one operation	Generally faster for many strings

7.7 Applications

1. Combining large amounts of text.
2. Generating reports and documents.
3. Preparing messages and notifications.
4. Improving Python program performance.
5. Comparing different programming methods.

7.8 Result

The program was implemented successfully using repeated string concatenation and the join() method. Both methods were validated by comparing their outputs. The execution times were measured to identify the more efficient method. Generally, join() is more efficient for combining a large number of strings.

Performance Analysis and Complexity Comparison

Performance analysis helps to identify how efficiently a Python program works. The execution time, number of operations, and memory usage are considered to compare different programming methods. Loops, recursion, and string operations are analysed to understand their performance and efficiency.

Results and Discussion

All seven Python tasks were implemented and tested successfully. The programs produced the expected outputs using decision-making statements, loops, string operations, functions, and recursion. Testing helped identify errors and improve program efficiency.

Conclusion

This assignment improved my understanding of Python programming concepts. I learned to use conditional statements, loops, strings, functions, and recursion to solve practical problems. I also understood the importance of testing, debugging, and improving program performance.

References

1. Python Official Documentation – https://docs.python.org/3/
2. Python Programming Tutorials – W3Schools.
3. Python Programming Concepts – GeeksforGeeks.
4. Course notes and classroom materials.

Appendix – Python Source Code and Execution Evidence

This appendix contains the Python source code for all seven tasks. It also includes the program execution outputs and screenshots captured from Visual Studio Code. These materials provide evidence that the programs were implemented and tested successfully.

Appendix – Python Source Code and Execution Evidence

1. Appendix A – Task 1 to Task 2

Python source code and execution screenshots for electricity subsidy calculation and OTP verification.

2. Appendix B – Task 3 to Task 5

Python source code and execution screenshots for vehicle registration validation, string concatenation, and functions with parameters.

3. Appendix C – Task 6 to Task 7

Python source code and execution screenshots for recursion, performance analysis, and complexity comparison.